INTERNSHIP | PROJECT REPORT

CYBERSECURITY &

ETHICAL HACKING

Task-3: Web Application Security

SUBMITTED TO	ApexPlanet Software Pty Ltd
PREPARED BY	Bikila Giragn Garba
ID	IAPSPL2648570
EMAIL	Bikila.girag@gmail.com
LINKEDIN	https://www.linkedin.com/in/bikila-giragn-827499241
GITHUB	https://github.com/bikgir

ADDIS ABABA, ETHIOPIA

November 2026

WEB APPLICATION SECURITY TESTING REPORT

Metasploitable 2 Vulnerable Lab Environment

Target: 10.0.0.6 | Assessment type: Authorized laboratory security testing

1. Executive Summary

This report documents a controlled security assessment of the Metasploitable 2 virtual machine (10.0.0.6). The assessment focuses on exposed services and web application functionality in the authorized laboratory environment. The supplied assessment material includes screenshots from DVWA demonstrating several security-testing modules.

The integrated evidence covers Brute Force, Command Execution, SQL Injection, File Upload, and Reflected Cross-Site Scripting (XSS). The supplied material does not contain a CSRF screenshot or network-service scan output, so those areas remain pending verification rather than being reported as confirmed findings.

2. Assessment Details

Assessment target	Metasploitable 2
Target IP address	10.0.0.6
Testing platform	Kali Linux (lab workstation)
Assessment environment	Isolated/authorized virtual lab
Web testing areas	DVWA modules evidenced in the supplied screenshots
Assessment date	November 07/2026
Tester	Bikila Giragn Garba

3. Objectives

- Identify network-accessible services and their versions on the target.
- Review exposed services for known weaknesses and insecure configurations.
- Test applicable web application inputs for SQL injection and cross-site scripting.
- Review authentication and other sensitive application functions for common weaknesses.
- Record evidence, assess risk, and recommend practical remediation.

4. Scope and Rules of Engagement

In scope: the Metasploitable 2 host at 10.0.0.6 and applications/services intentionally provided by the lab. Testing must remain within the isolated lab network and must not target public or third-party systems. Do not perform destructive actions, persistence, denial-of-service testing, or access to data outside the exercise.

Out of scope: other hosts, production systems, and any network not explicitly authorized. Evidence should record the exact application page, test input, response, and screenshot for each test.

5. Methodology and Tools

Phase	Tool / technique	Purpose
Discovery	ip a; ping; Nmap	Confirm lab connectivity and identify reachable services.
Enumeration	nmap -sV -sC 10.0.0.6	Identify open ports, service versions, and standard service information.
Vulnerability review	nmap --script vuln 10.0.0.6	Check for potential issues reported by applicable Nmap NSE scripts.
Web testing	Browser / Burp Suite; DVWA	Manually assess application behavior and security controls.
Evidence and reporting	Screenshots; test notes	Preserve reproducible evidence and document remediation.

6. Test Cases and Findings

The following sections integrate the supplied screenshots from the second document into the corresponding security-testing modules. Where screenshots are available, the evidence is presented under the relevant module. Where evidence is absent, the status is recorded as pending rather than assumed.

6.1 Brute Force

The supplied evidence contains two screenshots of the DVWA Brute Force module, including browser/Burp testing context. These screenshots are retained as evidence for the module. A final severity should be assigned only after confirming the exact authentication behavior, rate limiting, lockout controls, and impact.

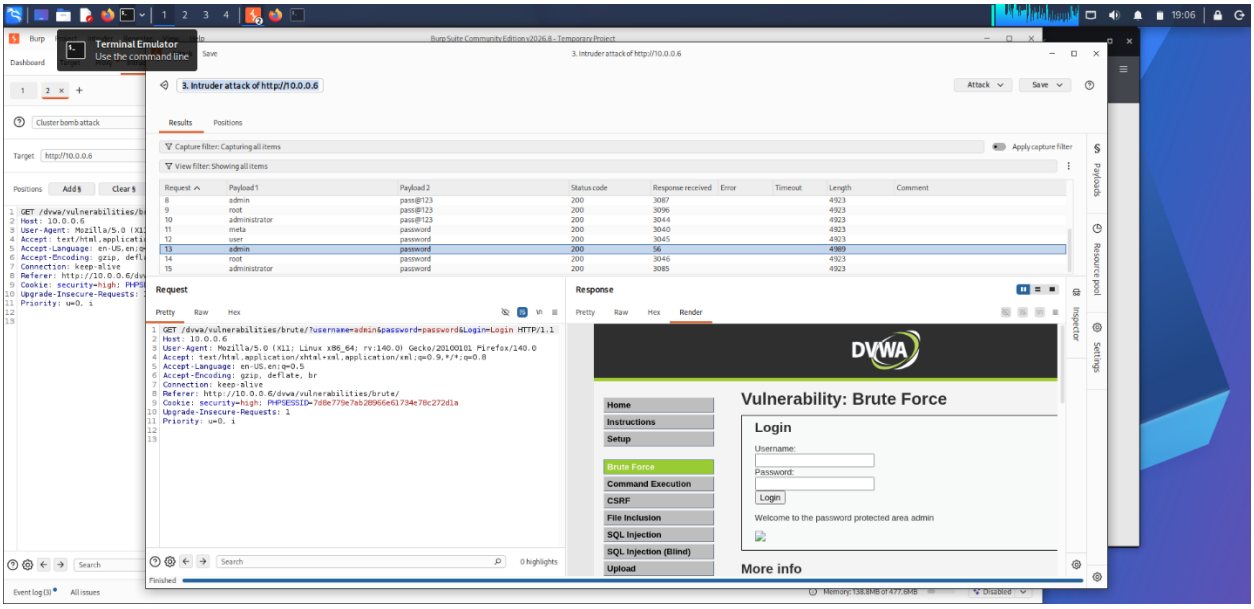


Figure 1. Brute Force module evidence — supplied screenshot.

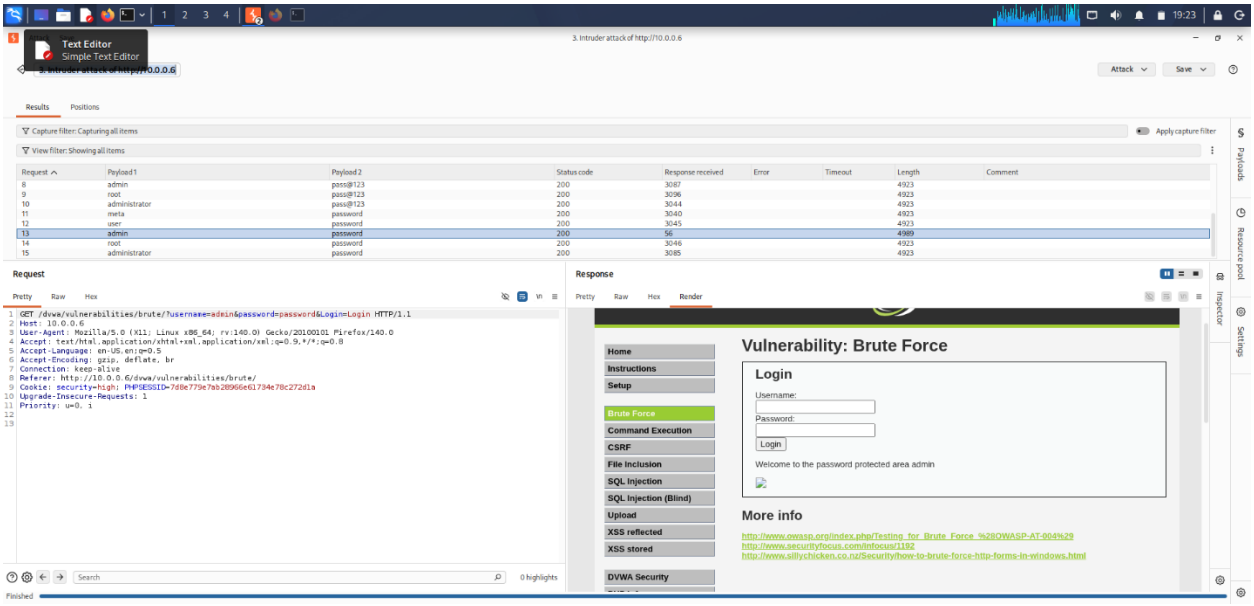


Figure 2. Brute Force module evidence — supplied screenshot.

6.2 Command Execution

The supplied screenshots show the DVWA Command Execution module and visible command/output content. This evidence should be used to document whether user-controlled input is reaching an operating-system command interpreter. Final severity depends on the verified command-execution capability and its impact within the authorized lab.

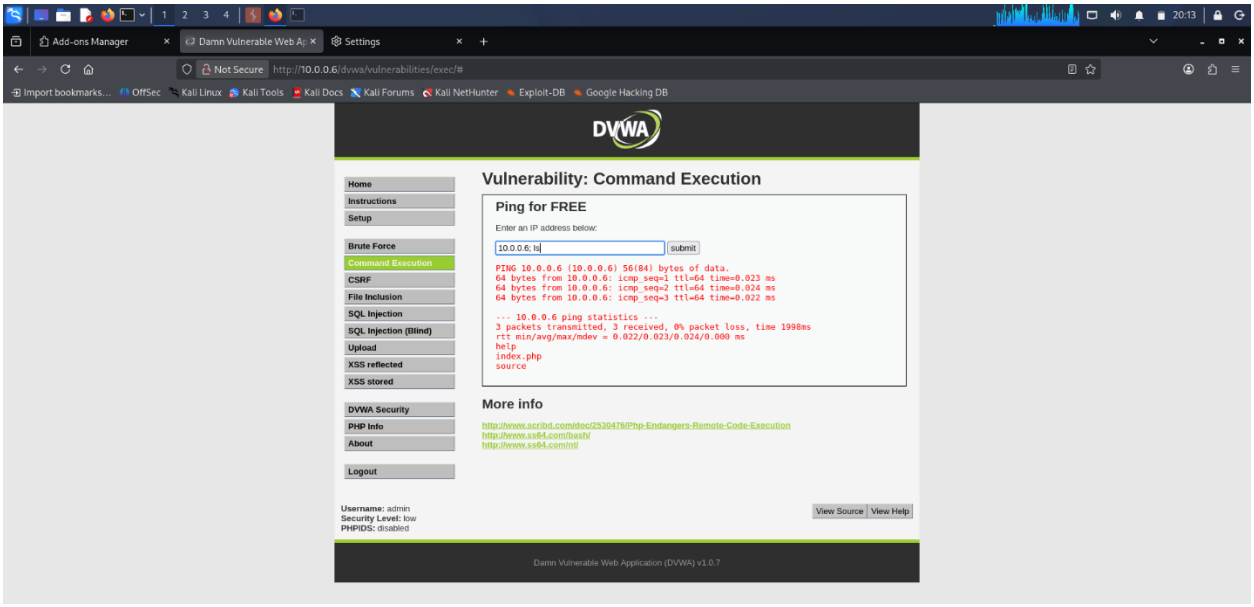


Figure 1. Command Execution module evidence — supplied screenshot.

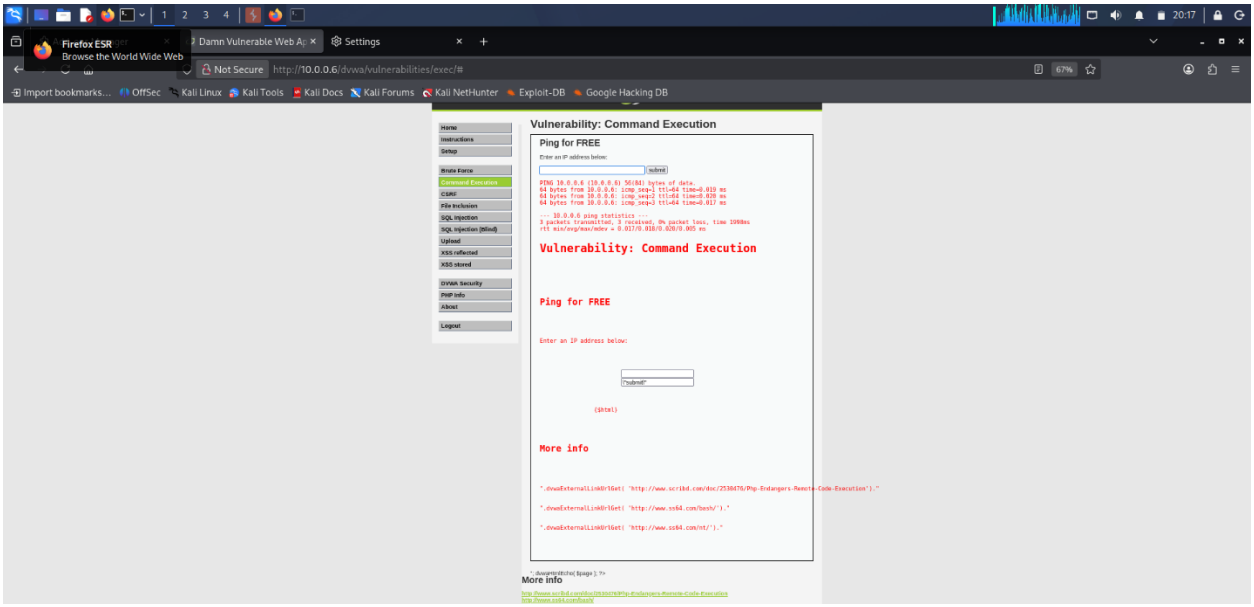


Figure 2. Command Execution module evidence — supplied screenshot.

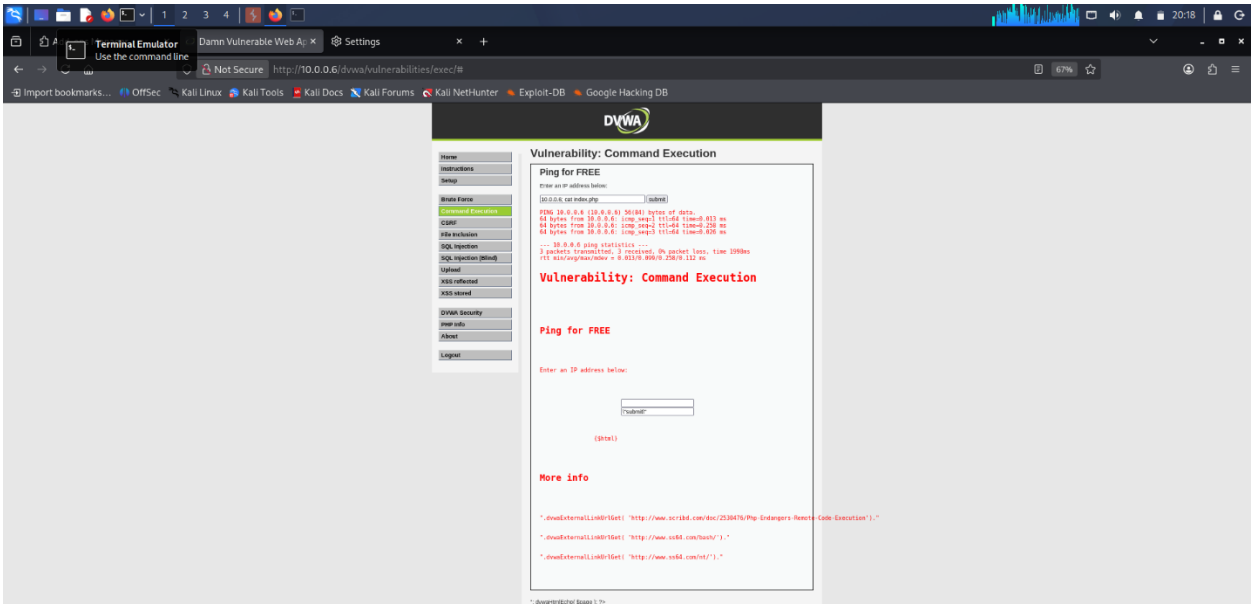


Figure 3. Command Execution module evidence — supplied screenshot.

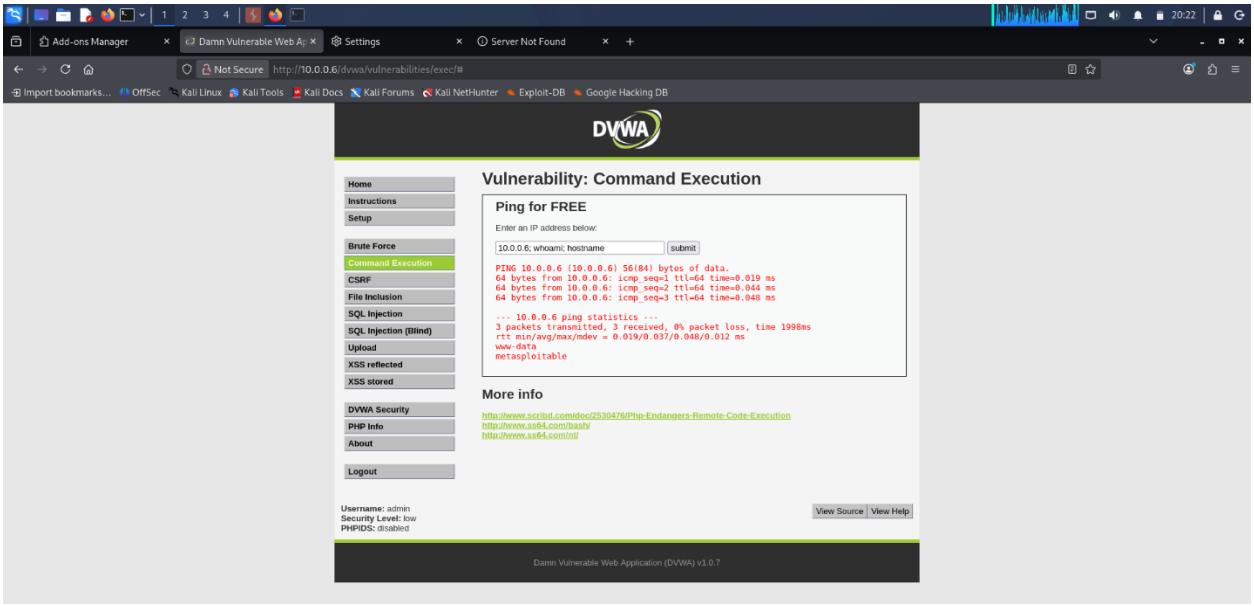


Figure 4. Command Execution module evidence — supplied screenshot.

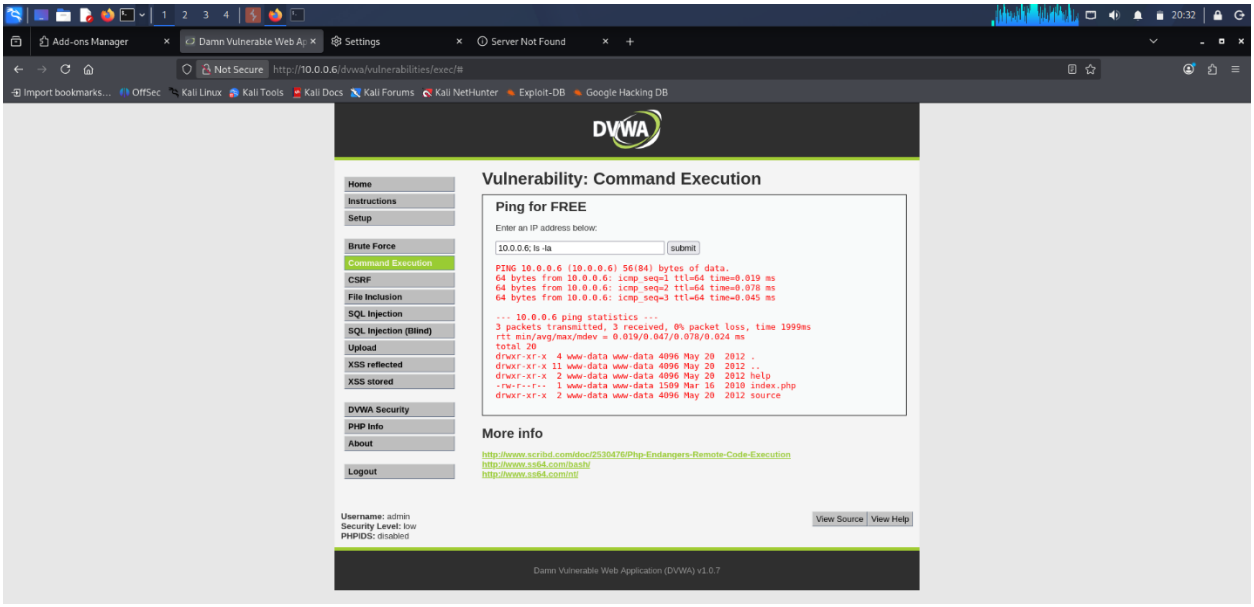


Figure 5. Command Execution module evidence — supplied screenshot.

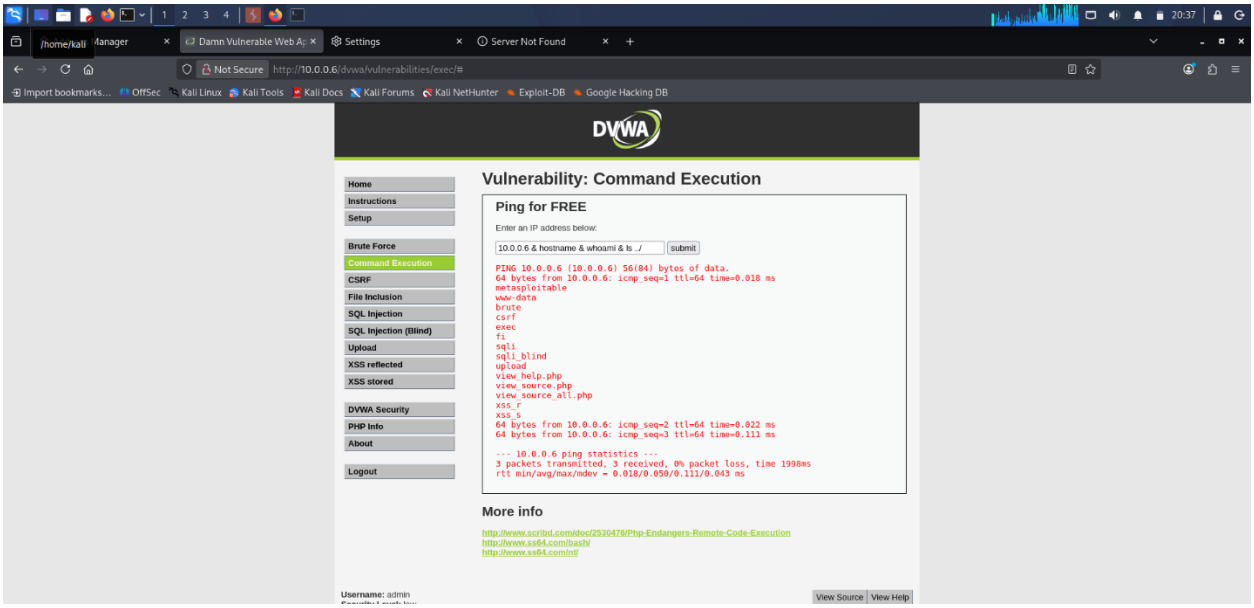


Figure 6. Command Execution module evidence — supplied screenshot.

Status: Evidence supplied; final vulnerability validation and severity: [TBD].

6.3 Information Disclosure / PHP Information

One supplied screenshot displays a PHP information page. This is relevant as an information-disclosure observation because detailed runtime and configuration information can assist further testing. The exact security impact should be determined from the configuration values visible in the supplied evidence.



Figure 7. PHP information page — supplied screenshot.

6.4 SQL Injection (SQLi)

Objective: determine whether user-controlled input is incorporated into a database query without safe parameterization. The supplied screenshots show the DVWA SQL Injection module, including test input and resulting page content. These screenshots provide module-specific evidence for the report.

Secure expected behavior: input is handled as data and does not alter SQL syntax or query behavior. Applications should use prepared statements/parameterized queries and server-side input validation.

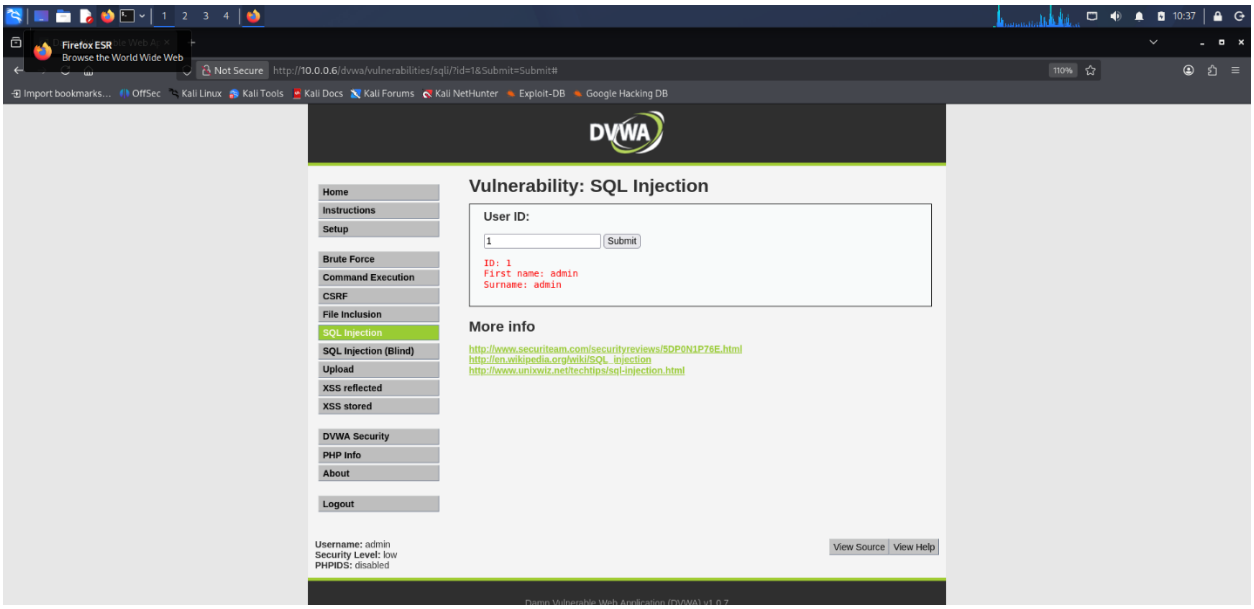


Figure 8. SQL Injection module evidence — supplied screenshot.

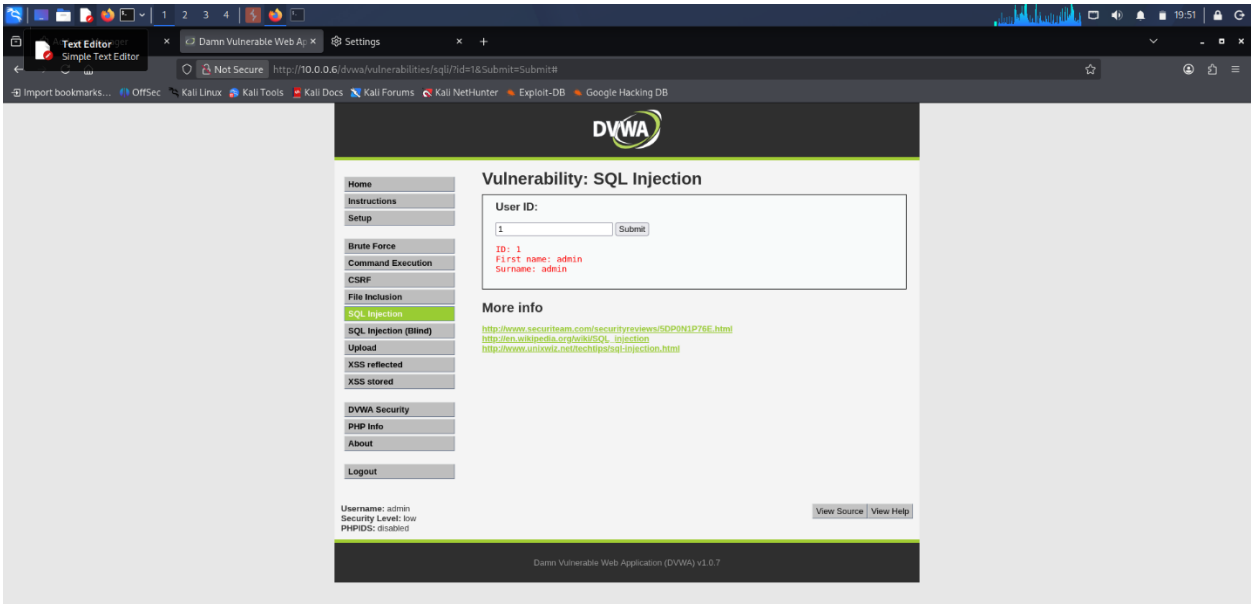


Figure 9. SQL Injection module evidence — supplied screenshot.

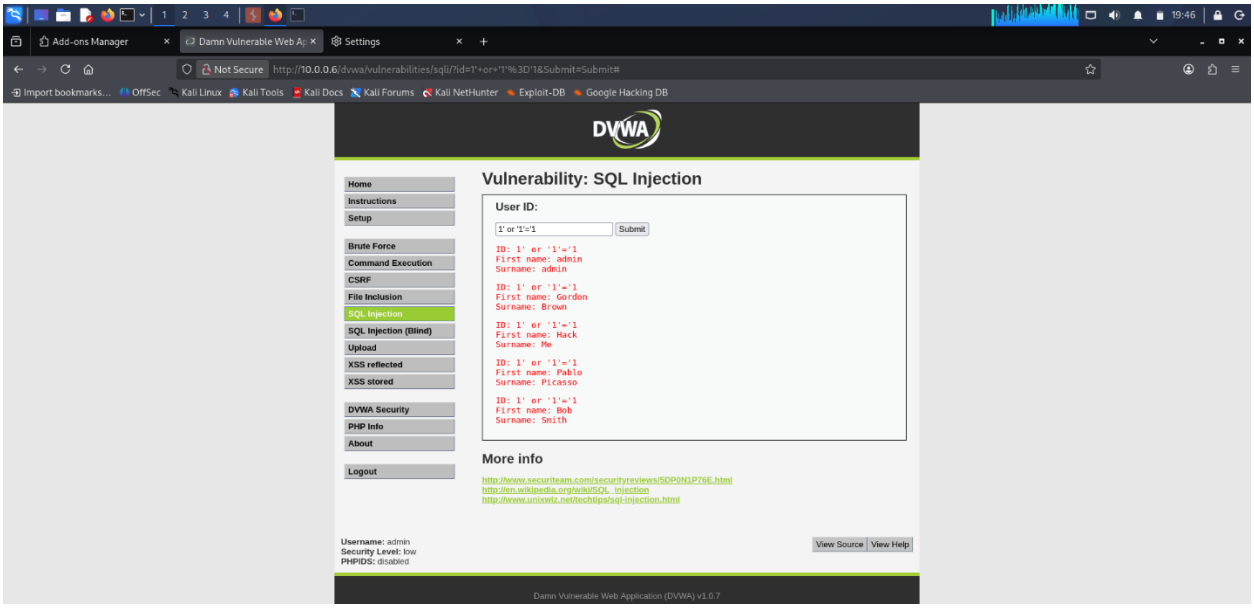


Figure 10. SQL Injection module evidence — supplied screenshot.

6.5 File Upload

The supplied screenshots show the DVWA File Upload module and its resulting page behavior. The module should be assessed for extension/content validation, storage location, execution permissions, filename handling, and server-side restrictions. The screenshots are included as the evidence supplied with the assessment materials.

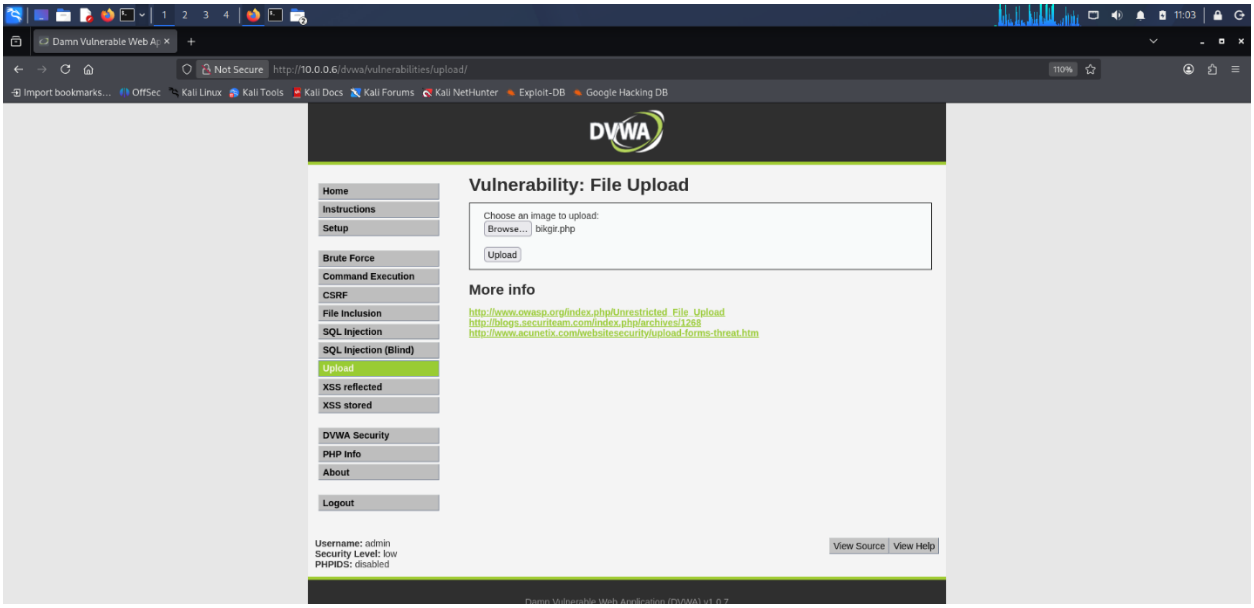


Figure 1. File Upload module evidence — supplied screenshot.

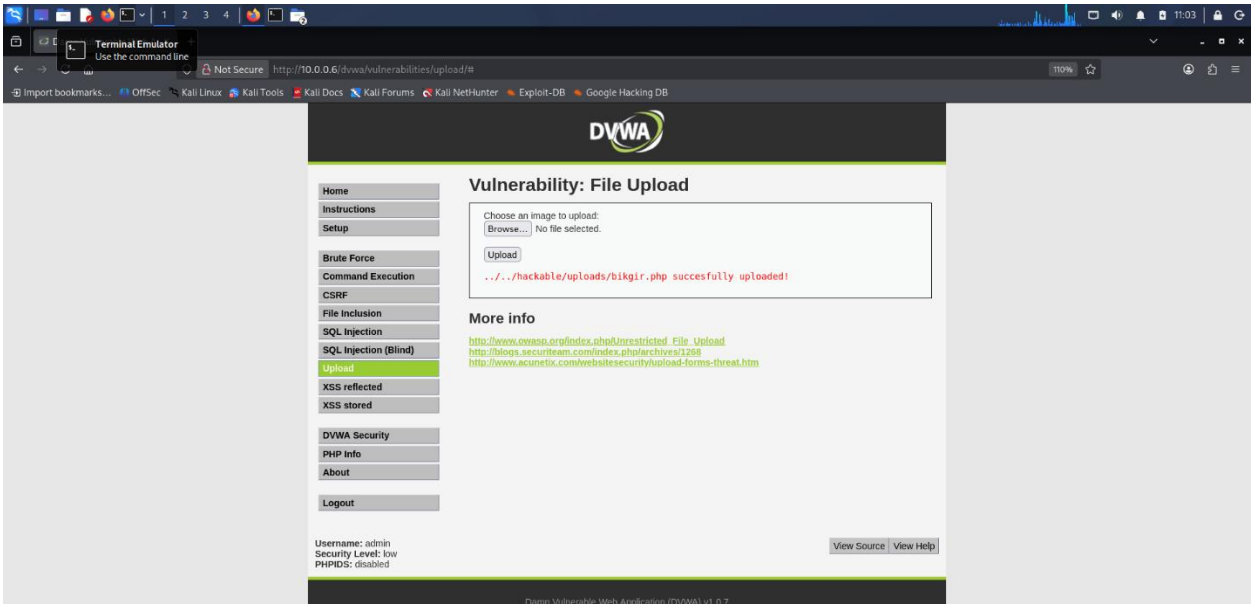


Figure 2. File Upload module evidence — supplied screenshot.

6.6 Cross-Site Scripting (XSS)

Objective: determine whether user-supplied content is returned to a browser without context-appropriate output encoding. The supplied screenshot shows the DVWA Reflected Cross-Site Scripting (XSS) module with browser execution behavior visible. This is evidence that the XSS test case should be recorded and validated against the exact input and response.

Secure expected behavior: untrusted input is safely encoded or sanitized for its output context and does not execute as script. Risk should be based on reach, persistence, affected users, and demonstrated impact.

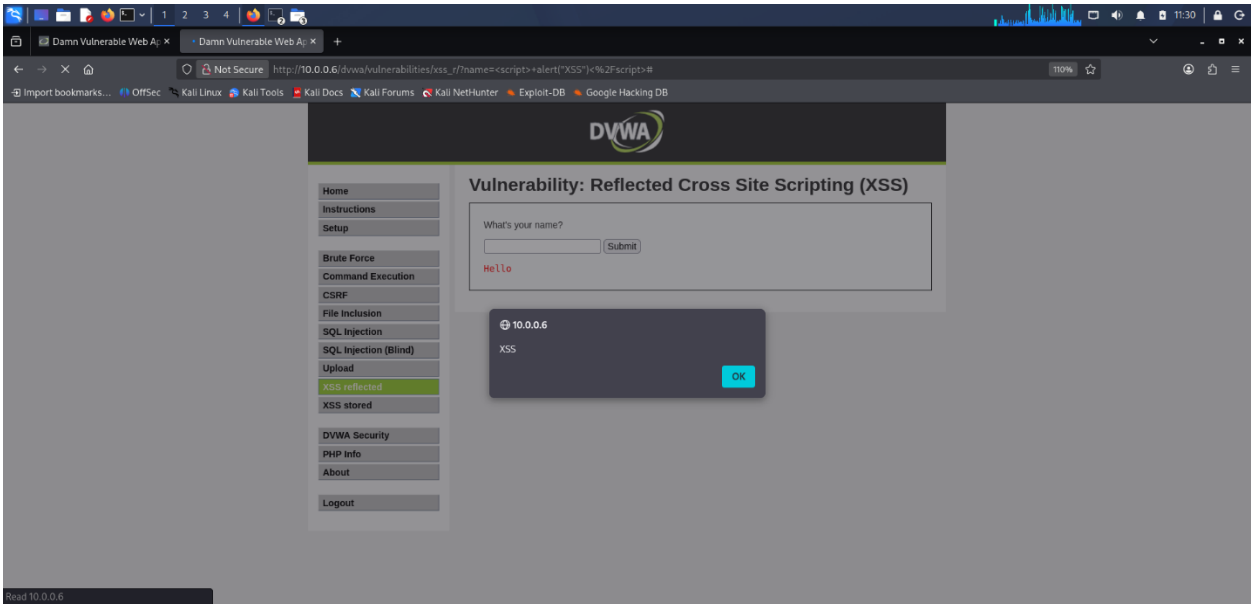


Figure 13. Reflected Cross-Site Scripting (XSS) module evidence — supplied screenshot.

6.7 Cross-Site Request Forgery (CSRF)

Objective: determine whether a sensitive state-changing action can be triggered without a valid anti-CSRF control. No CSRF screenshot was supplied in the evidence document. Therefore, this test remains not tested in this integrated report.

Evidence required: capture the legitimate request, presence or absence of a CSRF token, and the application's response to a missing or invalid token. Sensitive state-changing requests should require a validated, session-bound anti-CSRF token and appropriate SameSite cookie protections.

Status: Not tested — no supplied screenshot/evidence.

6.8 Network Service Enumeration

The original task document specifies Nmap service/version enumeration using `nmap -sV -sC 10.0.0.6` and, where appropriate in the isolated lab, `nmap --script vuln 10.0.0.6`.

7. Risk Rating Criteria

Critical: direct, severe compromise with broad impact. High: significant unauthorized access, data exposure, or code execution. Medium: meaningful weakness requiring specific conditions or user interaction. Low: limited impact or difficult exploitation. Informational: observation without a demonstrated security impact. Do not assign a final severity solely from a scanner label; validate the issue and consider impact and likelihood.

9. Recommendations

- Use prepared statements/parameterized queries for all database operations and validate input server-side.
- Apply context-aware output encoding; use a well-maintained framework and a restrictive Content Security Policy as defense in depth.
- Protect state-changing requests with unpredictable, session-bound CSRF tokens; validate tokens server-side and configure SameSite cookies.
- Apply rate limiting, account lockout or progressive delays, and strong authentication controls to defend against brute-force attacks.
- Avoid passing untrusted input directly to operating-system commands; use safe APIs, strict allow-lists, and least-privilege execution.
- Restrict PHP information disclosure and avoid exposing unnecessary runtime/configuration details in production.
- Validate uploaded files by type and content, store them outside executable web paths where possible, and enforce safe permissions and filenames.
- Disable unnecessary services and restrict management interfaces to trusted lab hosts.
- Patch vulnerable services, remove default credentials, enforce strong authentication, and review service configurations.

10. Conclusion

This combined report consolidates the original security-testing structure with the screenshots supplied in the accompanying evidence document. The screenshots provide module-specific evidence for Brute Force, Command Execution, PHP Information, SQL Injection, File Upload, and Reflected XSS. Because the supplied materials do not include complete test notes, scan output, or enough detail to independently establish final severity and exploitability for every module, the report intentionally marks those ratings

as TBD. CSRF and network-service enumeration remain pending because corresponding evidence was not supplied.